
Web Development and Design

Lecture 08: WebE Process: Construction

Construction

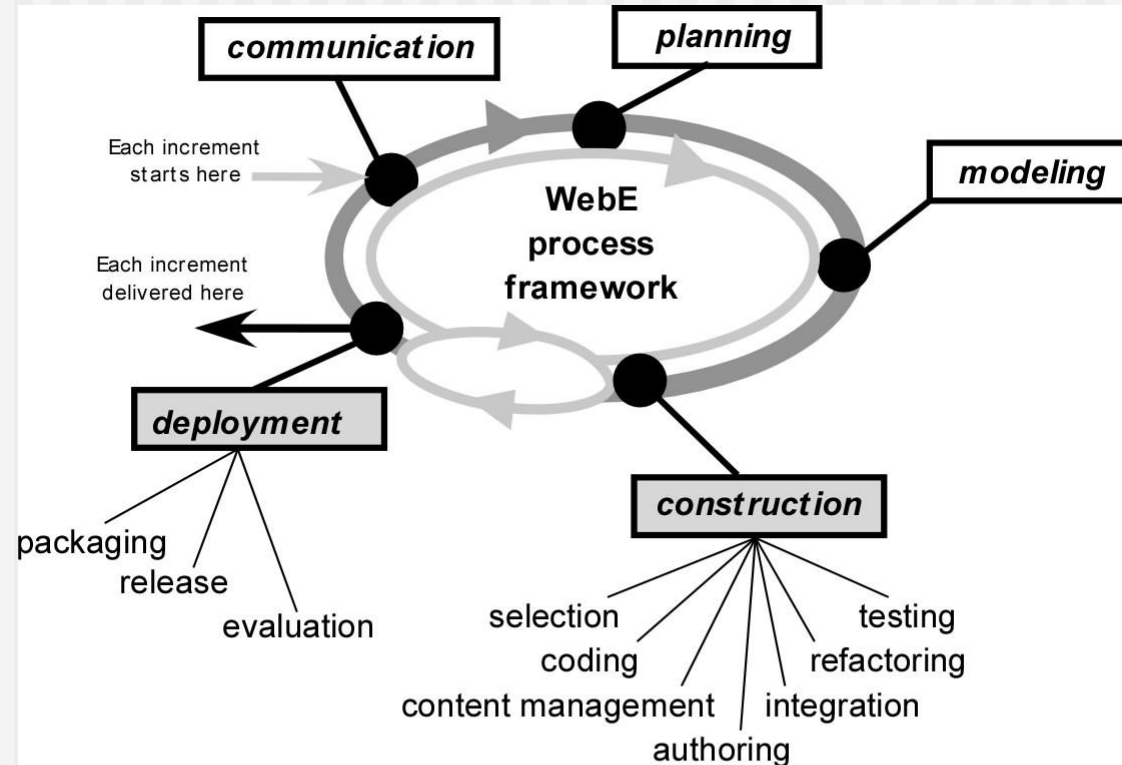


Figure 12-1 Construction and Deployment in the WebE Process Flow

WebE Process: Construction

- **Construction:** builds all informational and functional components that must be delivered for a WebApp increment.
- WebE tools and technology are applied to construct the WebApp that has been modeled.
 - Once the WebApp increment has been constructed, a series of rapid tests are conducted to ensure that errors in design (i.e., content, architecture, interface, navigation) are uncovered.
 - Additional testing addresses other WebApp characteristics.

WebE Process: Construction

- Encompasses a set of tasks that lead to an operational WebApp that is ready for delivery to end users:
 - **Selection** involves the identification of relevant preexisting components (or objects within components) that can be reused within the proposed design.
 - **Coding** covers the adaptation of existing components or creation of new components and may involve the direct creation of HTML or scripting-language source code or the automatic generation of code using an intermediate design representation of the component to be built.

WebE Process: Construction

- **Content management** involves the creation, migration, and structuring of content. This encompasses content creation, implementation of database schemas, or conversion of legacy content into, say, XML.
- **Authoring** involves the integration of raw content with the graphic design (layout) and the mapping of the content into the screens and pages.

WebE Process: Construction

- **Integration** comprises the linking of the code, content, and presentation into the final components to be released.
- **Refactoring** is an iterative action that “polishes” the implemented components to improve their structure and clarity and remove redundant code (ex: unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate content and data structures).
- **Testing** involves the verification that the various components and objects are correct.

Construction: Preparation Principles

- Before you create a single element of content, design a single Web page or write one line of code, be sure you:
 - Pick a language that meets the needs of the component to be built and the environment in which it will operate.
 - Select an environment that provides tools that will make your work easier.
 - Create a set of unit tests that will be applied once the component you create is completed.

Construction: Selection Principles

- As you select existing, reusable components and objects, be sure you
 - Take into account the constraints of the technical environment.
 - Match the components to the information and functional environments.
 - Consider the skills and knowledge of both the developers and likely maintainers.
 - Consider issues of IP (Intellectual Property), the proprietary nature of the components, and whether they are portable.

Construction: Coding Principles

- As you begin writing code, be sure you
 - Write code that is self-documenting.
 - Constrain your algorithms by following structured programming practices.
 - Select data structures that will meet the needs of the design.
 - Understand the functional architecture and create interfaces that are consistent with it.
 - Keep conditional logic as simple as possible and ensure it is testable.
 - Adopt coding styles that aid in readability (e.g., select meaningful identifier names and follow other local coding standards).

Construction: Content Management Principles

- As content is managed, be sure you
 - Select data structures that will meet the needs of the design.
 - Understand the information architecture and create content and navigational structures that are consistent with it.
 - Ensure consistency in the formats and data structures.
 - Avoid reliance on proprietary data formats.
 - Treat your content as publishable material (means prioritizing clarity, structure, and user experience)—not as software (Focuses on functionality).

Construction: Authoring Principles

- As you create Web pages (or templates for pages), be sure you
 - Continually consider issues of usability.
 - Remember to address issues of accessibility.
 - Understand how your users *will* react, not how *you want* them to react.
 - Learn from competitors.

Construction: Integration Principles

- As you integrate your components and objects, be sure you:
 - Keep backups – preferably in some form of version control system. You need to be able to rewind the WebApp to earlier versions.
 - Look for component interface mismatches or inconsistencies.
 - Take the opportunity to identify components that need refactoring.

Construction: Refactoring and Testing Principles

- **Refactoring principles.** As you refactor your WebApp be sure you
 - Refactor often, and in small steps, when the opportunity arises Make sure the implementation communicates the design in an obvious way.
- **Testing principles.** After you've completed your first components, be sure you
 - Conduct a walkthrough when appropriate.
 - Perform unit tests and correct errors you've uncovered.
 - Select tests that are most likely to locate errors rather than to hide them.

Testing WebApps

- Testing is the process of exercising a WebApp with the intent of finding (and ultimately correcting) errors.
- Tests must be design to uncover errors in WebApps that are implemented in:
 - different operating systems
 - browsers [or other interface devices such mobile phones]
 - hardware platforms
 - communications protocols

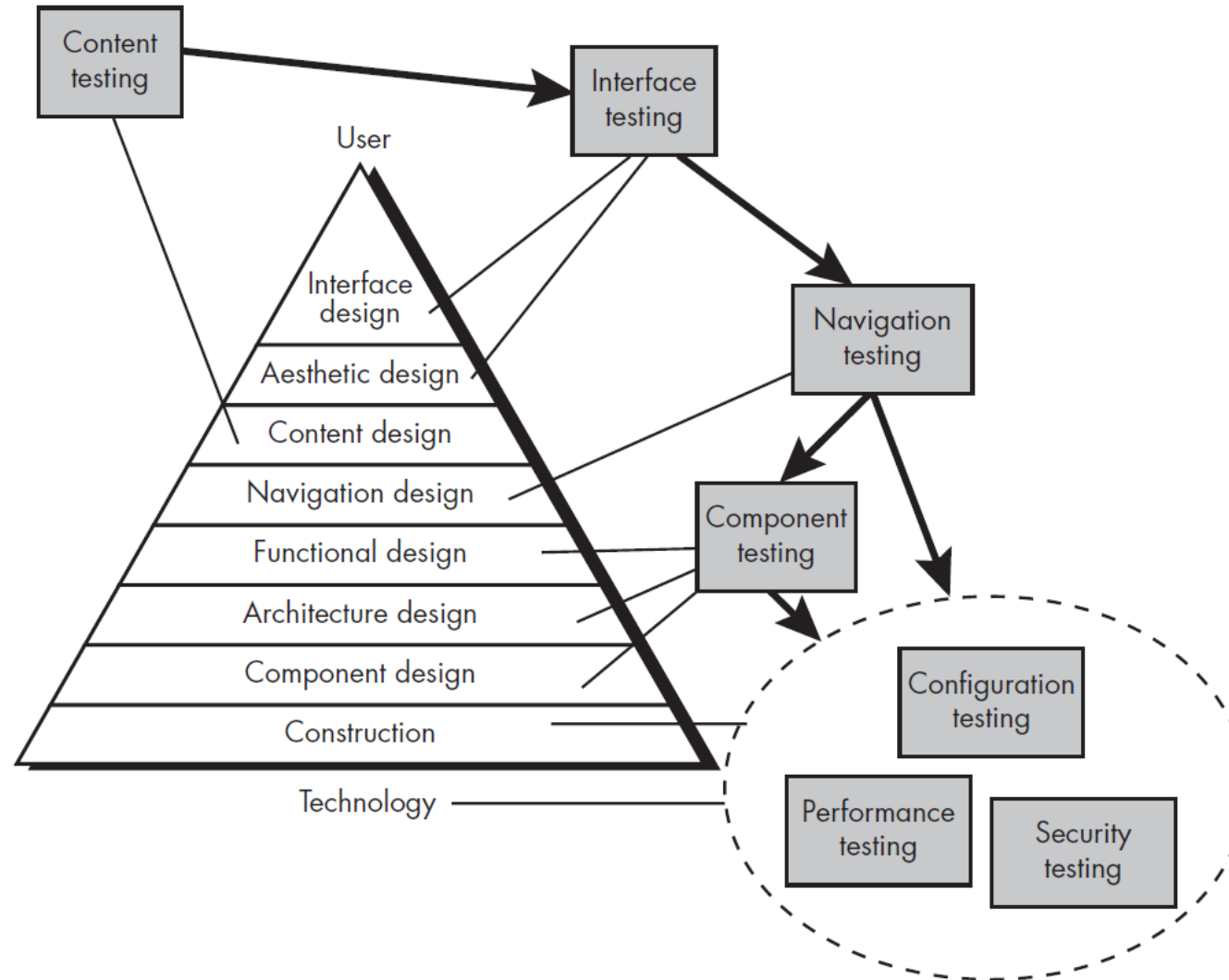
Testing Strategy

1. The content model for the WebApp is reviewed to uncover errors.
2. The interface model is reviewed to ensure that all use cases have been accommodated.
3. The design model for the WebApp is reviewed to uncover navigation errors.
4. The user interface is tested to uncover errors in presentation and/or navigation mechanics.
5. Selected functional components are unit tested.
6. Navigation throughout the architecture is tested.

Testing Strategy

7. The WebApp is implemented in a variety of different environmental configurations and is tested for compatibility with each configuration.
8. Security tests are conducted in an attempt to exploit vulnerabilities in the WebApp or within its environment.
9. Performance tests are conducted.
10. The WebApp is tested by a controlled and monitored population of end users.

The Testing Process



Content Testing

- Content testing has three important objectives:
 - to uncover syntactic errors (e.g., typos, grammar mistakes) in text-based documents, graphical representations, and other media,
 - to uncover semantic errors (i.e., errors in the accuracy or completeness of information) in any content object presented as navigation occurs, and
 - to find errors in the organization or structure of content that is presented to the end user.

Content Testing - Checklist

- Is the information up to date and factually accurate?
- Is the information concise and to the point?
- Is the layout of the content object easy for the user to understand?
- Can information embedded within a content object be found easily?
- Have proper references been provided for all information derived from other sources?
- Is the information presented consistent internally and consistent with information presented in other content objects?
- Can the content be interpreted as being offensive or misleading, or does it open the door to litigation?
- Does the content infringe on existing copyrights or trademarks?
- Does the content contain internal links that supplement existing content? Are the links correct?
- Does the aesthetic style of the content conflict with the aesthetic style of the interface?

User Interface: Testing Strategy

- Interface features are tested to ensure that design rules, aesthetics, and related visual content are available to the user without error.
- Each interface mechanism is tested within the context of a use case or navigation pathway for a specific user category.
- The interface is tested within a variety of environments (e.g., operating systems, browsers) to ensure that it will be compatible.

User Interface – Testing specific elements (1)

- When a user interacts with a WebApp, the interaction occurs through one or more interface mechanisms. Each mechanism must be tested:
 - **Links.** Navigation mechanisms that link the user to some other content object or function.
 - **Forms.** A structured document containing blank fields that are filled in by the user.
 - **Client-side scripting.** A list of programmed commands in a scripting language (e.g., JavaScript) that handle information input via forms or other user interactions.
 - **Dynamic HTML.** Provides access to content objects that are manipulated on the client side using scripting or cascading style sheets (CSSs).
 - **Client-side pop-up windows.** Small windows that pop up without user interaction.
 - **Server-side scripts.** Black-box tests are conducted with an emphasis on data integrity and script processing once validated data has been received. In addition, performance testing can be conducted.

User Interface – Testing specific elements (2)

- When a user interacts with a WebApp, the interaction occurs through one or more interface mechanisms. Each mechanism must be tested:
 - **Streaming and push content.** *Streaming content* is encountered when material (usually audio or video) is downloaded in a manner that allows it to be displayed while it is still being downloaded (rather than having to wait for the entire content to be downloaded). *Push content* is encountered when content objects are downloaded automatically from the server side rather than waiting for a request from the client side. Both streaming and push content present testing challenges.
 - **Cookies.** A block of data sent by the server and stored by a browser as a consequence of a specific user interaction. The content of the data is WebApp-specific (e.g., user identification data or a list of items that have been selected for purchase by the user).
 - **Application-specific interface mechanisms.** Include one or more “macro” interface mechanisms such as a shopping cart, credit card processing, or a shipping cost calculator.

Usability Testing

- Usability testing can occur at a variety of different levels of abstraction:
 1. the usability of a specific interface mechanism (e.g., a form) can be assessed
 2. the usability of a complete Web page (encompassing interface mechanisms, data objects, and related functions) can be evaluated, or
 3. the usability of the complete WebApp can be considered.

Usability Test Categories

- **Interactivity.** Are interaction mechanisms (e.g., pull-down menus, buttons, pointers) easy to understand and use?
- **Layout.** Are navigation mechanisms, content, and functions placed in a manner that allows the user to find them quickly?
- **Readability.** Is text well written and understandable? Are graphic representations intuitive and easy to understand?
- **Aesthetics.** Do the layout, color, typeface, and related characteristics lead to ease of use? Do users “feel comfortable” with the look and feel of the WebApp?

Usability Test Categories

- **Display characteristics.** Does the WebApp make optimal use of screen size and resolution?
- **Time sensitivity.** Can important features, functions, and content be used or acquired in a timely manner?
- **Personalization.** Does the WebApp appropriately tailor itself to the specific needs of different user categories or individual users?

Usability Evaluation: Checklist

- Is the system usable without continual help or instruction?
- Do the rules of interaction help a knowledgeable user to work efficiently?
- Do interaction mechanisms become more flexible as users become more knowledgeable?
- Has the system been tuned to the physical and social environment in which it will be used?
- Are users aware of the state of the system? Do users know where they are at all times?
- Is the interface structured in a logical and consistent manner?
- Are interaction mechanisms, icons, and procedures consistent across the interface?
- Does the interaction anticipate errors and help the user correct them?
- Is the interface tolerant of errors that are made?
- Is the interaction simple?

Compatability Testing

- WebApps operate in complex (and often unpredictable) environments
 - Different browsers, screen resolutions, operating systems, plug-ins, access bandwidths, etc.
- Serious errors can be caused by obscure combinations
- Most common problem is deterioration in usability:
 - Download speeds may become unacceptable
 - Missing plug-ins may make content unavailable
 - Browser differences can change page layout or legibility
 - Forms may be improperly organized.

Component-Level Testing

- *Component-level testing*, also called *function testing*, focuses on a set of tests that attempt to uncover errors in WebApp functions.
- Applies the following test-case design methods:
 - **Equivalence partitioning**
 - **Boundary value analysis**
 - **Path testing**

Selecting Components to Test

- Which functionality in the Web site is most critical to its purpose?
- Which areas of the site require the heaviest database interaction?
- What types of problems would cause the most complaints or the worst publicity?
- What areas of the site will be the most popular?
- What aspects of the site have the highest security risks?

Configuration Testing

- Configuration variability and instability are important factors that make Web engineering a challenge.
 - Hardware, operating system(s), browsers, storage capacity, network communication speeds, and a variety of other client-side factors are difficult to predict for each user.
- The job of configuration testing is to test a set of probable client-side and server-side configurations to ensure that the user experience will be the same on all of them and to isolate errors that may be specific to a particular configuration.

Testing Strategy

- **Server-side.** configuration test cases are designed to verify that the projected server configuration [i.e., WebApp server, database server, operating system(s), firewall software, concurrent applications] can support the WebApp without error.
- **Client-side.** On the client side, configuration tests focus more heavily on WebApp compatibility with configurations that contain one or more permutations of the following components:
 - **Hardware.** CPU, memory, storage, and printing devices
 - **Operating systems.** Linux, Macintosh OS, Microsoft Windows, a mobile-based OS
 - **Browser software.** FireFox, Internet Explorer, Safari, Mozilla/Netscape, Opera, and others
 - **User interface components.** Active X, Java applets, and others
 - **Plug-ins.** QuickTime, RealPlayer, and many others
 - **Connectivity.** Cable, DSL, regular modem, industry-grade connectivity (e.g., T1 lines)

Security and Performance Testing

- Security and performance testing address the three distinct elements of the WebApp infrastructure
 - the server-side environment that provides the gateway to Internet users
 - the network communication pathway between the server and the client machine
 - the client-side environment that provides the end user with a direct interface to the WebApp.
- **Security testing** focuses on unauthorized access to WebApp content and functionality along with other systems that cooperate with the WebApp on the server side.
- **Performance testing** focuses on the operating characteristics of the WebApp and on whether those operating characteristics meet the needs of end users.

Performance Testing

■ Objectives:

- Does the server response time degrade to a point where it is noticeable and unacceptable?
- At what point (in terms of users, transactions, or data loading) does performance become unacceptable?
- What system components are responsible for performance degradation?
- What is the average response time for users under a variety of loading conditions?
- Does performance degradation have an impact on system security?
- Is WebApp reliability or accuracy affected as the load on the system grows?
- What happens when loads that are greater than maximum server capacity are applied?
- What is the impact of poor performance on company revenues?

- **Load testing** determines how the WebApp and its server-side environment will respond to various loading conditions.

Thank You